Université Paris-Dauphine (PSL)
Bachelor's degree, Year 3 — Computer Science, Mathematics and Decision (IM2D)

Internship Report

JSON Data Generation

by an LLM

the case of the `not` keyword

Student
Zein Miled

Internship supervisor
Dario Colazzo

Co-supervisor
Nour Ben Ali

July 31, 2026

Contents

Overview **2**

1 Internship topic **2**

2 The model studied **2**

3 The schema corpus **2**

4 First experiment: can the model generate valid data? **3**
4.1 Method . . . 3
4.2 Few-shot examples . . . 3
4.3 Error categories . . . 4

5 Second experiment: is `not` really responsible for the errors? **4**
5.1 Method: ablation . . . 4
5.2 Does few-shot really improve the understanding of `not`? . . . 6
5.3 A direct recognition test . . . 6

6 An unexpected finding: unsatisfiable schemas **6**

7 Building a reference base **7**

8 Third experiment: helping the model with relevant examples (RAG) **7**
8.1 Principle . . . 7
8.2 How the pipeline works . . . 7
8.3 Results . . . 7

9 Conclusion **8**
Future work . . . 9

Code and data **9**

Overview

As a third-year student in the IM2D bachelor's program at Université Paris-Dauphine, I carried out this work during my internship, under the supervision of Dario Colazzo and with the co-supervision of Nour Ben Ali. This report presents all the work done since the start of the internship, from the first experiments to the most recent results.

1 Internship topic

This work focuses on language models, the artificial intelligence systems that can generate text (such as ChatGPT). I looked at a specific question: can a language model generate JSON data that follows a predefined set of rules?

The JSON format represents data in a structured way (for example, information about a person: name, age, city of residence). To check whether a JSON file is valid, a second document called a **schema** (JSON Schema) is used; it describes precisely the rules to follow: required fields, expected value types, and so on.

Among all these rules, I focused on one keyword, `not`, which expresses a prohibition. A schema can, for example, state that a city field must never take the value "Paris". This is a **negation** constraint. The goal of this work is to determine whether a language model actually manages to respect this kind of constraint, or whether it has specific difficulties with negation.

2 The model studied

I chose **Phi-2**, a language model developed by Microsoft. This choice has two justifications: first, its size, much smaller than that of the largest models (such as GPT-4), makes it easy to run; second, a smaller model is more likely to reveal the limits of its reasoning clearly.

3 The schema corpus

Evaluating the model required a large number of JSON schemas containing the `not` keyword. For this, I used an existing dataset, *jsonschemabench*, which gathers several thousand real-world schemas from various sources (GitHub, Kubernetes, the Washington Post, among others).

By filtering this dataset to keep only the schemas with at least one occurrence of `not`, I obtained **202 schemas**, spread across ten collections.

Collection	Schemas containing `not`
Github_hard	44
JsonSchemaStore	39
Github_medium	30
Github_ultra	23
WashingtonPost	23
Kubernetes	19
Github_easy	9
Github_trivial	8
Glaiveai2K	7
Snowplow	0
Total	**202**

4 First experiment: can the model generate valid data?

4.1 Method

For each schema, I asked the model to generate a valid JSON instance. Two settings were compared:

- **zero-shot**: only the schema is provided, with no example;
- **few-shot**: the schema comes with three examples meant to guide the model's answer.

Each answer is then checked automatically with a validation library, `jsonschema`. Figure 1 shows the results obtained on the Github_medium collection (30 schemas).

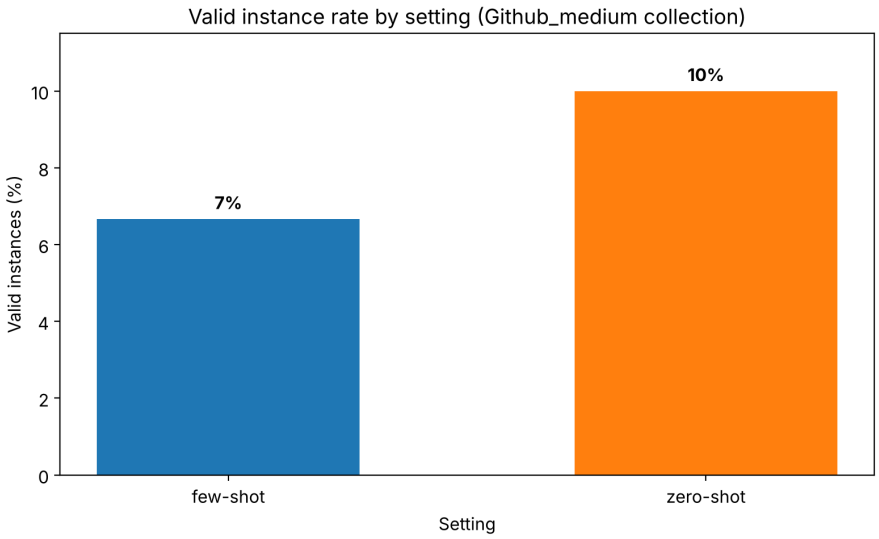


Figure 1: Valid instance rate in zero-shot and few-shot, on the Github_medium collection (30 schemas).

4.2 Few-shot examples

The three examples given to the model in the few-shot setting were chosen to stay neutral, that is, general enough not to steer the answer toward any particular schema of the test corpus. They illustrate three different uses of `not`: on an object property, on an array element, and on a string.

First example: negation on a required property.

```
Schema:   {"type":"object","not":{"required":["deprecated"]},
           "properties":{"title":{"type":"string"}}}
Instance: {"title":"hello"}
```

Second example: negation on an array element.

```
Schema:   {"type":"array","not":{"contains":{"const":"zzz"}}}
Instance: ["alpha","beta"]
```

Third example: negation on a string pattern.

```
Schema:   {"type":"string","not":{"pattern":"^xx_"}}
Instance: "kappa"
```

4.3 Error categories

The observed failures are of several different kinds, which led to a classification into five categories:

- **overflow**: the schema is too large for the model to process in one go;
- **truncated**: the answer stops before it is complete;
- **schema copy**: the model copies the schema instead of producing an instance;
- **wrong content**: the answer is valid JSON but breaks a schema rule other than `not`;
- **`not` violation**: all constraints are met except one: the forbidden value appears in the answer.

Figure 2 shows how these categories are distributed on the same collection, for each of the two settings.

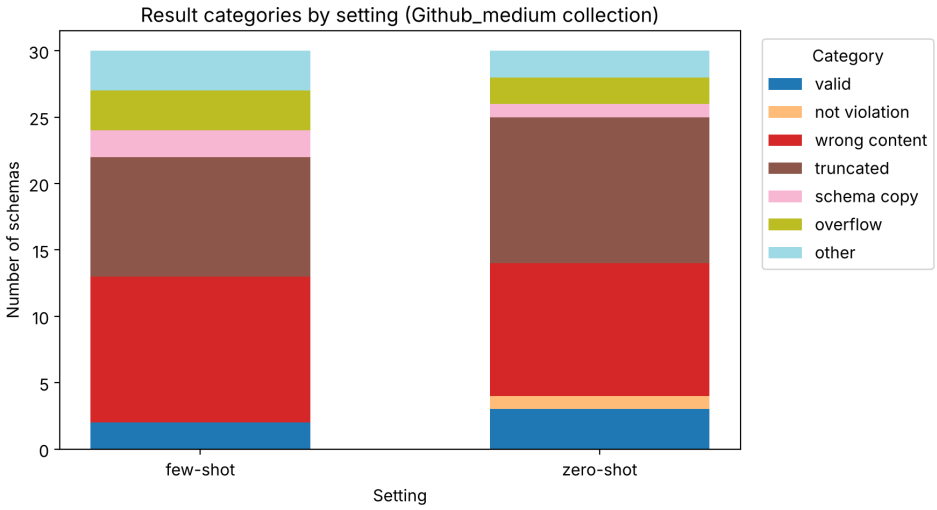


Figure 2: Distribution of result types in zero-shot and few-shot, on the Github_medium collection (30 schemas).

A large share of the errors simply comes from the schema exceeding the model's reading capacity (overflow), regardless of the `not` keyword. This observation motivated a second, more rigorous experiment, designed to isolate the specific effect of this constraint.

5 Second experiment: is `not` really responsible for the errors?

5.1 Method: ablation

To check whether `not` is indeed the source of the observed errors, I ran an **ablation** experiment: for each schema, a second version is created, identical except that the `not` is removed. The model's results are then compared on both versions.

When the model succeeds on the version without `not`, but its answer would have been invalid against the original version (because it contains exactly the forbidden value), the schema is considered **blocked by `not`**.

The success rate goes from 10/202 with `not` to 18/202 without it, nearly double (Figure 3).

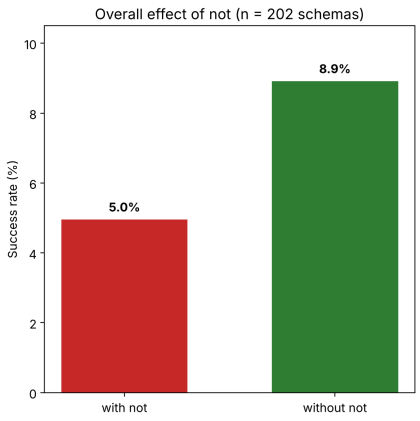


Figure 3: Success rate with and without `not`, over all 202 schemas.

Figure 4 breaks this result down by collection.

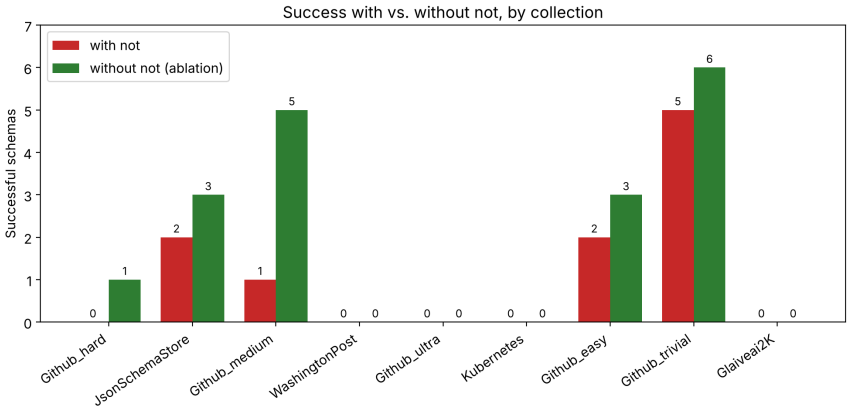


Figure 4: Number of successful schemas with and without `not`, by collection.

In addition, seven schemas were identified as blocked by this constraint alone. They are spread across three collections (Figure 5).



Figure 5: Number of schemas blocked specifically by `not`, by collection.

5.2 Does few-shot really improve the understanding of `not`?

A comparison between zero-shot and few-shot was also carried out on this set.

Switching to few-shot unlocks fifteen schemas, but makes four others fail that previously succeeded, a net gain of eleven schemas (Figure 6).

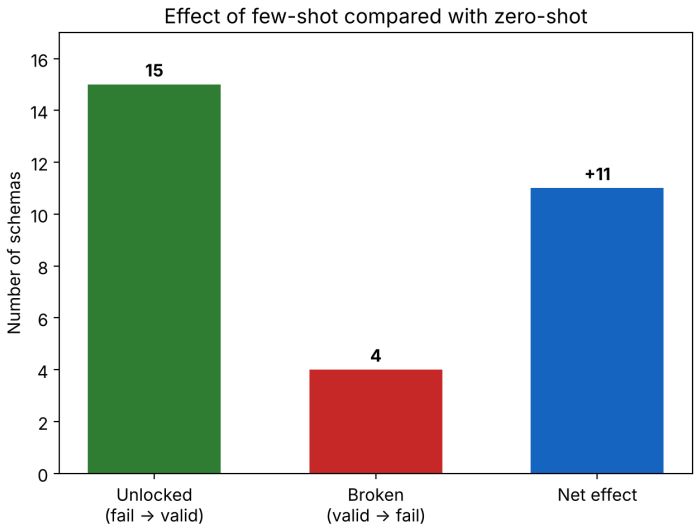


Figure 6: Effect of few-shot compared with zero-shot: unlocked and broken schemas.

However, of the fifteen unlocked schemas, only two belonged to the seven schemas identified as truly blocked by `not`. This suggests that few-shot mainly improves the overall formatting of the generated JSON, without significantly strengthening the understanding of negation.

5.3 A direct recognition test

To isolate the model's ability to handle negation even more clearly, a final test was run: instead of asking for a generation, the model is given a value directly and asked whether it is valid or not ("valid" or "invalid"), for ten cases involving `not`.

Of the ten values actually forbidden by the constraint, the model detected **none**, answering "valid" every time. This result, obtained under the simplest possible conditions, is the clearest sign of a real and specific difficulty of the model in reasoning about logical negation.

6 An unexpected finding: unsatisfiable schemas

Studying a family of seven schemas (named `calculate_area`) showed that the model systematically failed on them, whatever method was used. An exhaustive check, testing every possible combination of values, revealed the cause.

Five of these seven schemas turned out to be **logically unsatisfiable**: the schema requires both a property and its absence, so no instance, however correct, could ever satisfy it. This is therefore not a failure of the model, but a design flaw in the schema itself. The remaining two schemas, on the other hand, turned out to be satisfiable, as a valid solution was found after checking.

7 Building a reference base

Following these experiments, I built a dataset of 202 (schema, valid instance) pairs, intended to prepare the next stage of the project, presented in the following section. Each instance was systematically checked with the validator, both to make sure it conforms to the schema and to ensure it actually exercises the `not` constraint (rather than being a minimal instance that avoids it without really testing it).

For reasonably sized schemas, the instances were written by hand. For the largest schemas, some of which exceed 900,000 characters, a program was developed to build an instance automatically from the schema's rules, then fix it iteratively until a valid version was obtained.

At the end of this work, 96 % of the 202 instances have realistic, carefully written content, and the five unsatisfiable schemas are explicitly flagged as such in the base.

8 Third experiment: helping the model with relevant examples (RAG)

8.1 Principle

The last part of this work consisted in testing a method called **RAG** (retrieval-augmented generation). The idea: instead of always giving the model the same examples, each new schema is paired with the most similar examples found in the base of 202 schemas built earlier.

8.2 How the pipeline works

- each schema in the base is converted into a numerical representation (an **embedding**), which makes it possible to compare schemas with one another;
- for a new schema, the most similar examples are retrieved from the base (the **retriever**);
- a text (the **prompt**) is built from these examples and the target schema;
- the model generates an answer from this text;
- the validity of this answer is checked.

Two similarity methods were compared:

- **MiniLM**, a general-purpose model, sensitive to the meaning of the words used;
- **CodeBERT**, trained on source code, more sensitive to the structure of the schema.

8.3 Results

Several configurations were tested on a sample of twenty schemas (table below and Figure 7):

Configuration	Result
No RAG (zero-shot)	1/20 (5 %)
RAG with 3 examples, MiniLM	6/20 (30 %)
RAG with 1 example, MiniLM	11/20 (55 %)
RAG with 1 example, CodeBERT	11/20 (55 %)
Best combination found	**12/20 (60 %)**

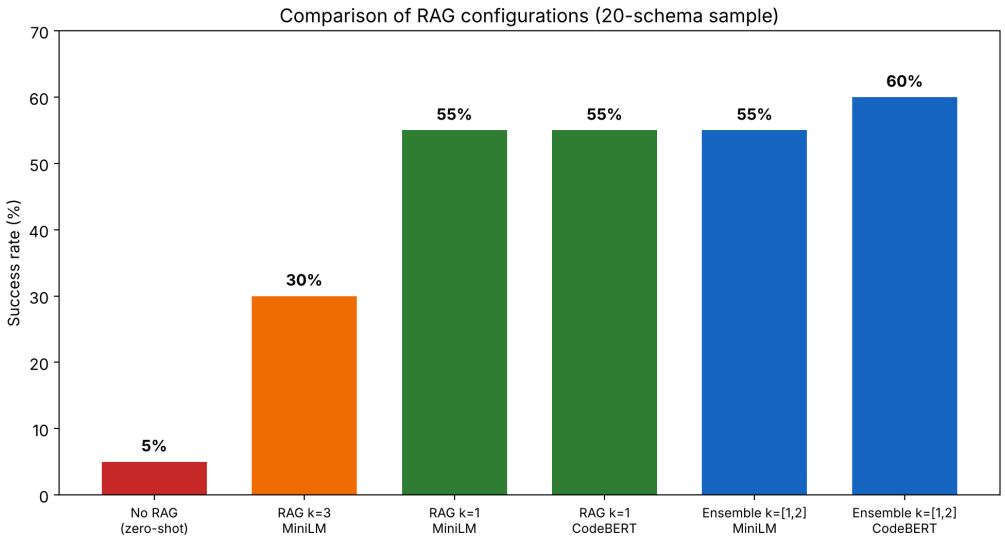


Figure 7: Comparison of the RAG configurations tested.

Several lessons emerge from these experiments:

- RAG improves the results considerably compared with providing no example at all (roughly a tenfold increase);
- providing a single example works better than providing two or three. This result, counter-intuitive at first sight, is explained by Phi-2's limited context window (2,048 tokens at most): the more examples are added, the less room is left for the model to finish its answer, hence a growing share of truncated answers;
- the structure-oriented embedding (CodeBERT) slightly outperforms the general-purpose one (MiniLM);
- changing the wording of the instructions given to the model does not improve the results, and can even make them worse when the text becomes too long;
- a careful count of all the observed errors shows that only about 35 % of them are actual `not` violations. The others have different causes (wrong data type, incorrect value, etc.). The `not` keyword remains a real difficulty, but in practice it is not the main cause of failure.

9 Conclusion

This work highlights a real difficulty for Phi-2 in handling negation constraints expressed with the `not` keyword. This conclusion rests on several converging experiments: the ablation study, the comparison between zero-shot and few-shot, and above all the direct recognition test, in which the model detected none of the ten violations presented.

This work also showed that a dataset, even one built from real-world sources, can contain logically contradictory schemas, which should be checked before evaluating a model on such a corpus.

Finally, the RAG method clearly improves the overall results, but without specifically fixing the difficulty with `not`: the number of errors of this kind remains similar across all the configurations tested.

Future work

As a follow-up to this work, it would be worth reproducing these experiments with a larger model, to determine whether the difficulty observed with `not` comes from the size of the model used or reflects a more general phenomenon.

Code and data

All the code and data for this project are available on GitHub:

https://github.com/zeinmil/witness-generation-bench